LAPORAN TUGAS INDIVIDU: PERANCANGAN RTL

SEQUENTIAL 2D CONVOLUTION ENGINE

EL4013 Perancangan Sistem VLSI

Disusun Oleh:

Brian A. Hadian

13520348

Sekolah Teknik Elektro dan Informatika (STEI)

Institut Teknologi Bandung

October 11, 2026

1 Bab I: Desain Sistem (System Architecture & Design)

1.1 Arsitektur FSMD & Transisi State

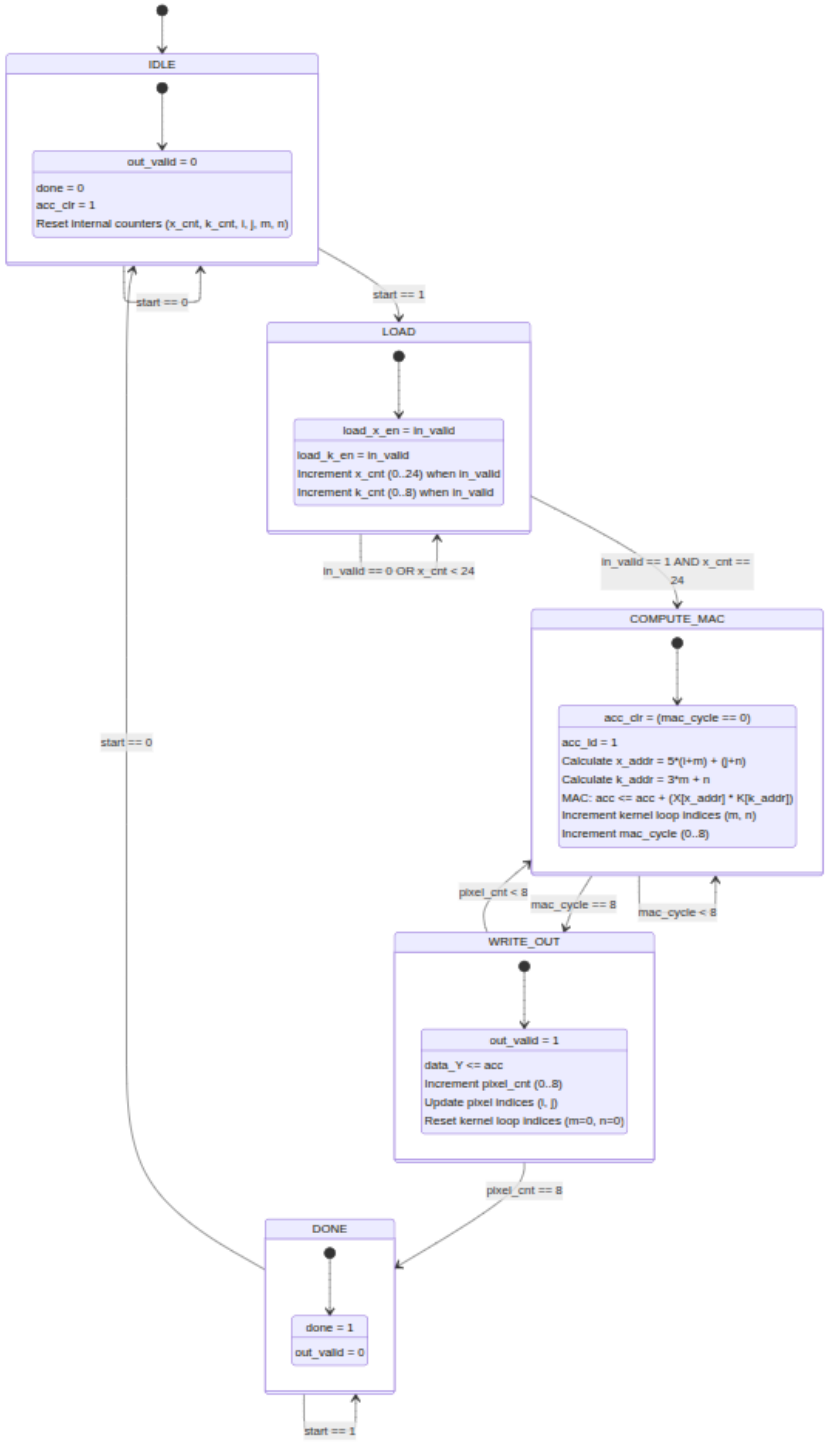


Figure 1: Diagram Transisi State FSM Control Unit

Sistem *Sequential 2D Convolution Engine* dirancang menggunakan pendekatan *Finite State Machine with Datapath* (FSMD). Unit kontrol terpisah dari unit jalur data (*data-*

path) untuk optimasi penggunaan area dengan cara *MAC sharing*. State terdiri dari 5 kondisi utama:

1. **IDLE**: Menunggu kemunculan sinyal `start`. Seluruh nilai diinisialisasi ke nilai nol.
2. **LOAD**: Membaca data masukan X (5×5) dan kernel K (3×3) secara sekuensial dan menyimpannya ke dalam register internal buffer saat `in_valid` bernilai HIGH.
3. **COMPUTE_MAC**: Melakukan 9 kali eksekusi perkalian-akumulasi $(X \times K)$ secara sekuensial per pixel $Y[i, j]$.
4. **WRITE_OUT**: Menyiapkan data keluaran $Y[i, j]$ pada `data_Y` dan melakukan output setiap `out_valid` selama 1 siklus clock positive edge.
5. **DONE**: Menandai penyelesaian seluruh komputasi 3×3 piksel keluaran dengan sinyal `done` bernilai HIGH.

1.2 Skematik Datapath & Pengalamatan



Figure 2: Skematik Tingkat Register Datapath & Antarmuka Kontrol (Sumber)

Datapath terdiri dari register file X (25×8-bit *unsigned*), register file K (9×8-bit *signed*), pengali 8u $\times$ 8s, dan akumulator 20-bit *signed*. Setiap *address* dipetakan dengan rumusan sebagai berikut:

$$\text{Addr}_X = 5 \times (i + m) + (j + n) \tag{1}$$

$$\text{Addr}_K = 3 \times m + n \tag{2}$$

2 Bab II: Kode Verilog RTL (Synthesizable Code)

Kode RTL dirancang (*fully synthesizable*) dan disimpan pada berkas `conv2d_engine.v`. Pemisahan antara unit FSM dan datapath diimplementasikan menggunakan blok `always` terpisah untuk transisi state kombinasional dan state sekuensial.

```
// FSM State Register & Synchronous Logic
always @(posedge clk or negedge rstn) begin
    if (!rstn)
        current_state <= S_IDLE;
```

```
    else
        current_state <= next_state;
end

// Single MAC Accumulation Execution
always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        acc_reg <= 20'sd0;
    end else if (current_state == S_COMPUTE_MAC) begin
        if (mac_cnt == 4'd0)
            acc_reg <= $signed({{3{product[16]}}, product});
        else
            acc_reg <= acc_reg + $signed({{3{product[16]}}, product});
    end
end
```

Listing 1: Potongan Kode Utama FSMD Sequential Convolution Engine

3 Bab III: Testbench & Strategi Pengujian

3.1 Metode Verifikasi

```
hadynata-IdeaPad-Gaming-3-15IAH7:tutorial-3 hadynata$ python models/golden_model.py
============================================================
      SEQUENTIAL 2D CONVOLUTION ENGINE - GOLDEN MODEL
============================================================

--- Input Matrix X (5x5, Unsigned 8-bit) ---
[[1 2 0 1 3]
 [2 1 3 2 0]
 [0 1 2 1 1]
 [1 3 0 2 2]
 [2 0 1 0 1]]

--- Filter Kernel K (3x3, Signed 8-bit) ---
[[ 1  0 -1]
 [ 1  0 -1]
 [ 1  0 -1]]

--- Element-wise Calculation Breakdown ---
Y[0,0] = (1*1) + (2*0) + (0*-1) + (2*1) + (1*0) + (3*-1) + (0*1) + (1*0) + (2*-1) =  -2 | Hex(20b): 0xFFFFE
Y[0,1] = (2*1) + (0*0) + (1*-1) + (1*1) + (3*0) + (2*-1) + (1*1) + (2*0) + (1*-1) =   0 | Hex(20b): 0x00000
Y[0,2] = (0*1) + (1*0) + (3*-1) + (3*1) + (2*0) + (0*-1) + (2*1) + (1*0) + (1*-1) =   1 | Hex(20b): 0x00001
Y[1,0] = (2*1) + (1*0) + (3*-1) + (0*1) + (1*0) + (2*-1) + (1*1) + (3*0) + (0*-1) =  -2 | Hex(20b): 0xFFFFE
Y[1,1] = (1*1) + (3*0) + (2*-1) + (1*1) + (2*0) + (1*-1) + (3*1) + (0*0) + (2*-1) =   0 | Hex(20b): 0x00000
Y[1,2] = (3*1) + (2*0) + (0*-1) + (2*1) + (1*0) + (1*-1) + (0*1) + (2*0) + (2*-1) =   2 | Hex(20b): 0x00002
Y[2,0] = (0*1) + (1*0) + (2*-1) + (1*1) + (3*0) + (0*-1) + (2*1) + (0*0) + (1*-1) =   0 | Hex(20b): 0x00000
Y[2,1] = (1*1) + (2*0) + (1*-1) + (3*1) + (0*0) + (2*-1) + (0*1) + (1*0) + (0*-1) =   1 | Hex(20b): 0x00001
Y[2,2] = (2*1) + (1*0) + (1*-1) + (0*1) + (2*0) + (2*-1) + (1*1) + (0*0) + (1*-1) =  -1 | Hex(20b): 0xFFFFF

--- Output Matrix Y (3x3, Signed 20-bit Decimal) ---
[[-2  0  1]
 [-2  0  2]
 [ 0  1 -1]]

--- Output Matrix Y (3x3, Signed 20-bit Hexadecimal) ---
0xFFFFE  0x00000  0x00001
0xFFFFE  0x00000  0x00002
0x00000  0x00001  0xFFFFF

============================================================
              HARDWARE TIMING & LATENCY ANALYSIS
============================================================
1. Data Loading Phase (Concurrent) : 25 Clock Cycles
   - X Buffer (25 Bytes)           : 25 Cycles (Active during in_valid)
   - K Buffer (9 Bytes)            : 9 Cycles (Active during in_valid)
2. MAC Computation Phase           : 81 Clock Cycles
   - Computed Pixels               : 9 elements
   - Sequential MACs/Pixel         : 9 MAC operations
3. Output Strobe Assertions        : 9 Cycles (1 pulse per Y[i,j])
4. State Transition Overhead       : 3 Cycles (IDLE/LOAD/DONE)
------------------------------------------------------------
TOTAL COMPUTATION LATENCY          : 109 Clock Cycles
============================================================
```

Figure 3: *Screenshot* Eksekusi Python Golden Model (`golden_model.py`)

Pengujian dilakukan menggunakan testbench Verilog (`tb_conv2d.v`) dengan input masukan matriks X (5×5) dan matriks kernel K (3×3). Hasil eksekusi RTL diverifikasi menggunakan *golden model* berbasis skrip Python (`golden_model.py`).

3.2 Matriks Masukan Kasus Uji

Masukan gambar X dan kernel filter K diset sebagai berikut:

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 & 3 \\ 2 & 1 & 3 & 2 & 0 \\ 0 & 1 & 2 & 1 & 1 \\ 1 & 3 & 0 & 2 & 2 \\ 2 & 0 & 1 & 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \tag{3}$$

4 Bab IV: Hasil Simulasi & Analisis Timing

4.1 Tabel Perbandingan Hasil Simulasi

Tabel 1 menunjukkan perbandingan nilai kalkulasi manual/Python model terhadap hasil simulasi RTL Verilog.

Table 1: Perbandingan Hasil Perhitungan Konvolusi $Y[i, j]$

Elemen Output	Ekspektasi (Decimal)	RTL Simulasi	Format Hex (20-bit)	Status
$Y[0,0]$	-2	-2	`0xFFFFE`	PASS
$Y[0,1]$	0	0	`0x00000`	PASS
$Y[0,2]$	1	1	`0x00001`	PASS
$Y[1,0]$	-2	-2	`0xFFFFE`	PASS
$Y[1,1]$	0	0	`0x00000`	PASS
$Y[1,2]$	2	2	`0x00002`	PASS
$Y[2,0]$	0	0	`0x00000`	PASS
$Y[2,1]$	1	1	`0x00001`	PASS
$Y[2,2]$	-1	-1	`0xFFFFF`	PASS

4.2 Analisis Timing & Gelombang Waveform

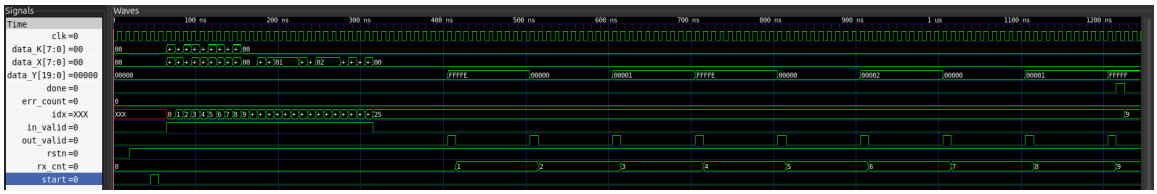


Figure 4: *Screenshot* Gelombang Simulasi Timing Waveform RTL

Berdasarkan hasil simulasi gelombang (*waveform*), transisi state berjalan dengan akurat:

- Pengisian buffer pada state `LOAD` membutuhkan 25 period clock bersamaan dengan sinyal `in_valid`.
- Setiap elemen $Y[i, j]$ membutuhkan 9 period clock akumulasi pada state `COMPUTE_MAC`.
- Sinyal `out_valid` bernilai HIGH selama 1 periode clock untuk setiap nilai $Y[i, j]$ yang akan di-*execute*.

- Sinyal `done` naik menjadi HIGH setelah seluruh 9 piksel Y selesai dihitung (total 109 periode clock).

5 Bab V: Kesimpulan & Pembahasan

5.1 Analisis Efisiensi Resource (1 MAC vs 9 MAC Parallel)

Penggunaan 1 unit MAC sekuensial memberikan efisiensi area silikon yang signifikan dibandingkan arsitektur paralel 9 MAC:

- **Penghematan Luas Area**: Mengurangi kebutuhan MAC (8u $\times$ 8s) dari 9 unit menjadi hanya 1 unit multiplier, menghemat estimasi area datapath hingga $\approx 88\%$ (8/9).
- **Trade-off Latensi**: Membutuhkan total 81 siklus komputasi MAC dibanding 9 siklus pada arsitektur paralel. Trade-off ini baik dimanfaatkan untuk aplikasi embedded/IoT yang memprioritaskan efisiensi *power* dan area.

5.2 Kesimpulan

RTL *Sequential 2D Convolution Engine* berbasis FSMD telah berhasil dirancang, disimulasikan, dan diverifikasi. Seluruh fungsi kontrol dan memori, perhitungan, dan hasil sinyal terbukti valid.